

Processi e Thread

Luca Spalazzi

l.spalazzi@staff.univpm.it

www.univpm.it/luca.spalazzi



- I processi e le operazioni sui processi
- I processi e Java
- I thread e le operazioni sui thread
- I thread e Java
- I processi e i thread Linux



Processi e Threads

I PROCESSI E LE OPERAZIONI SUI PROCESSI

Processo = Programma in esecuzione.

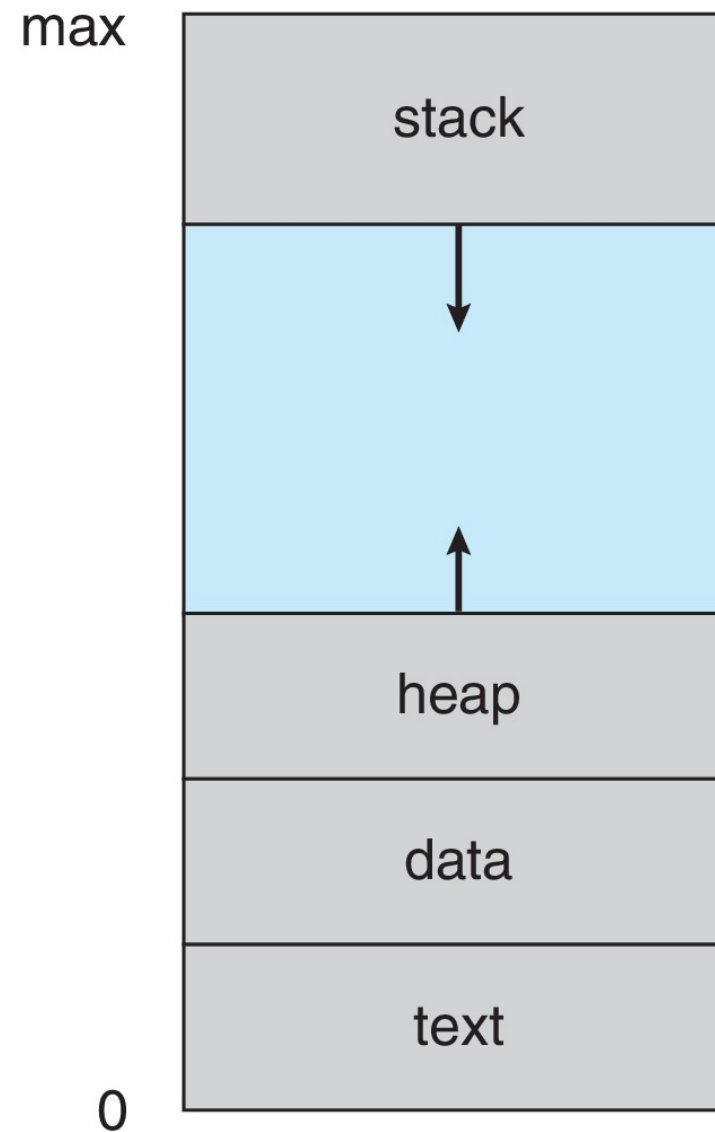
Formalmente:

$$P = (C, S)$$

- P : processo
 - C : codice eseguibile
 - S : stato dell'esecuzione
- fanno parte dello stato di esecuzione di un processo:
- program counter
 - altri registri della CPU
 - stato del processo
 - stack
 - data section
 - lista dei file aperti
 -



Un Processo in Memoria



Un Programma C in Memoria

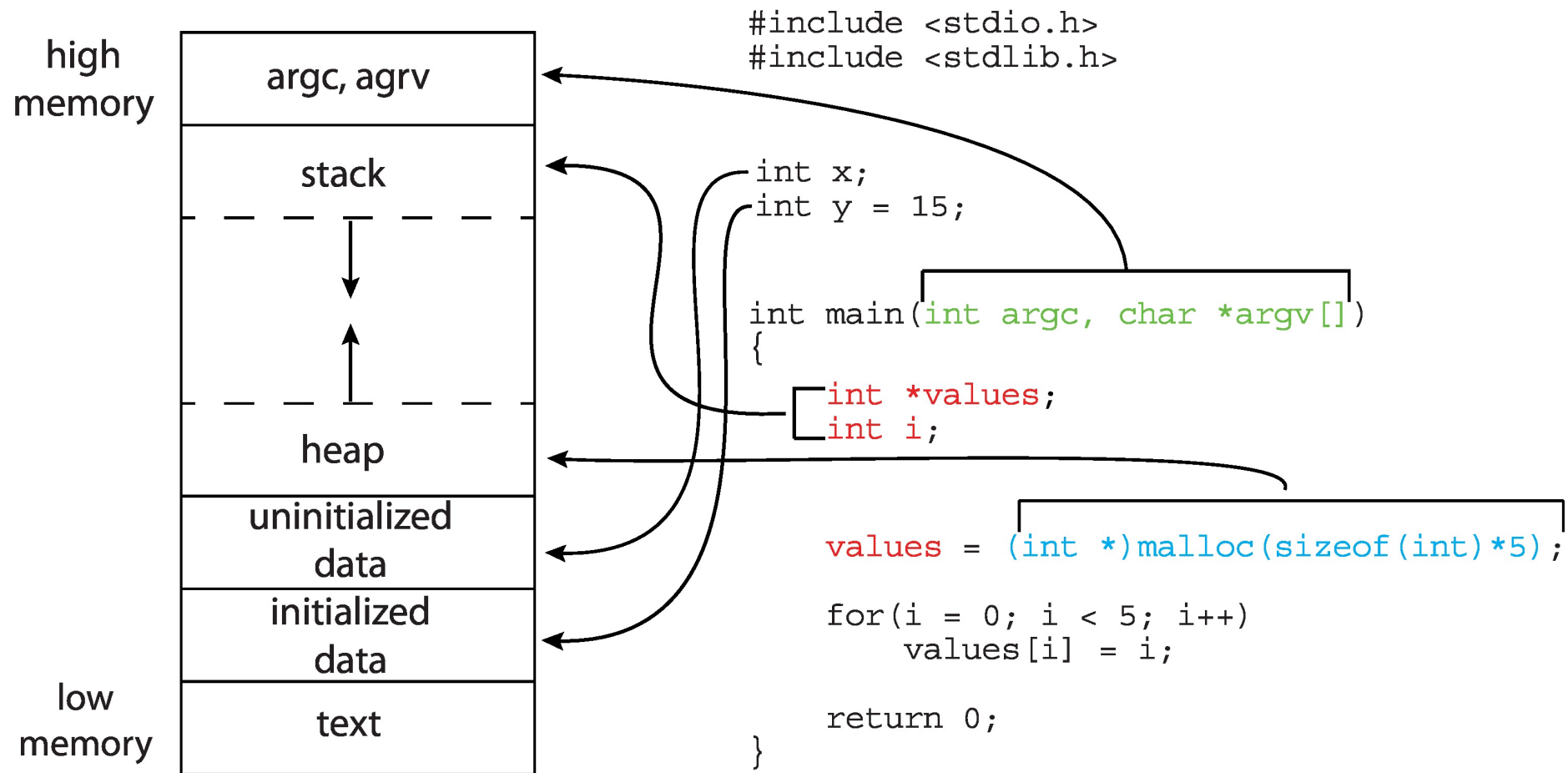
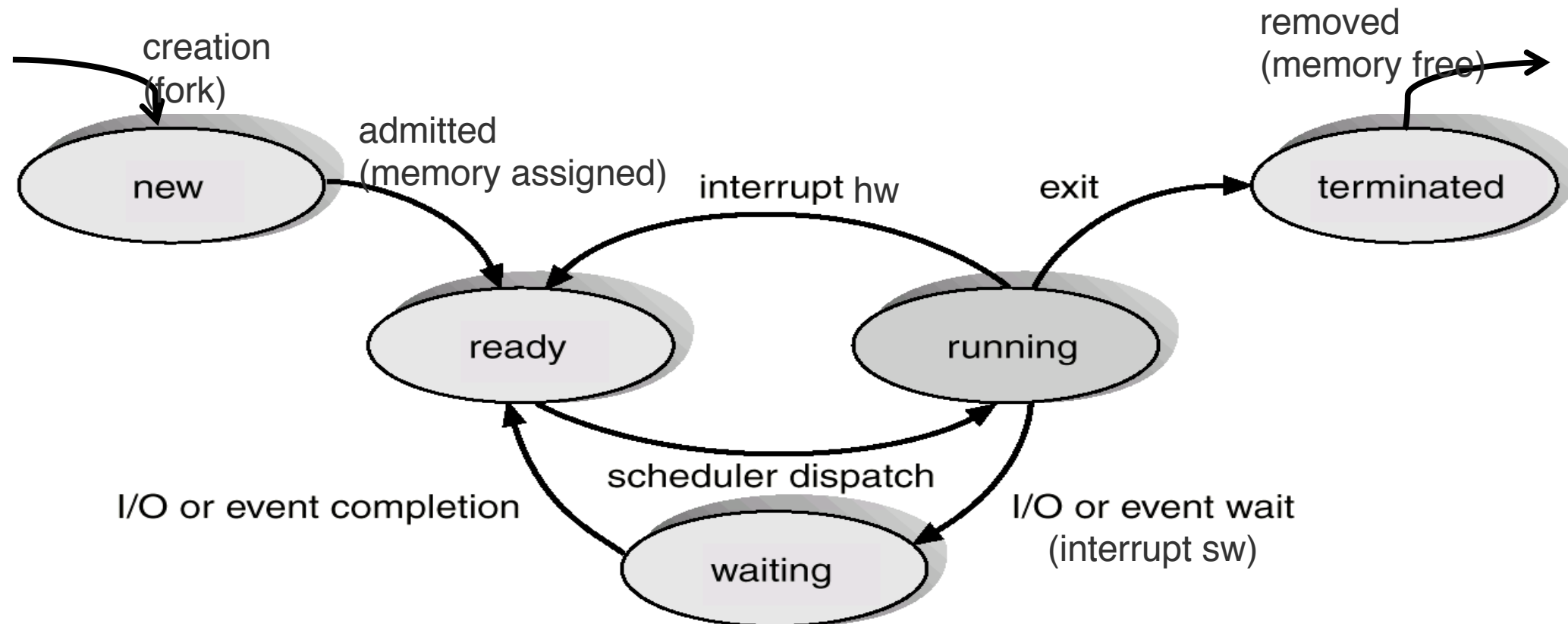


Diagramma di Stato di un Processo



Process Control Block (PCB) o descrittore di processo

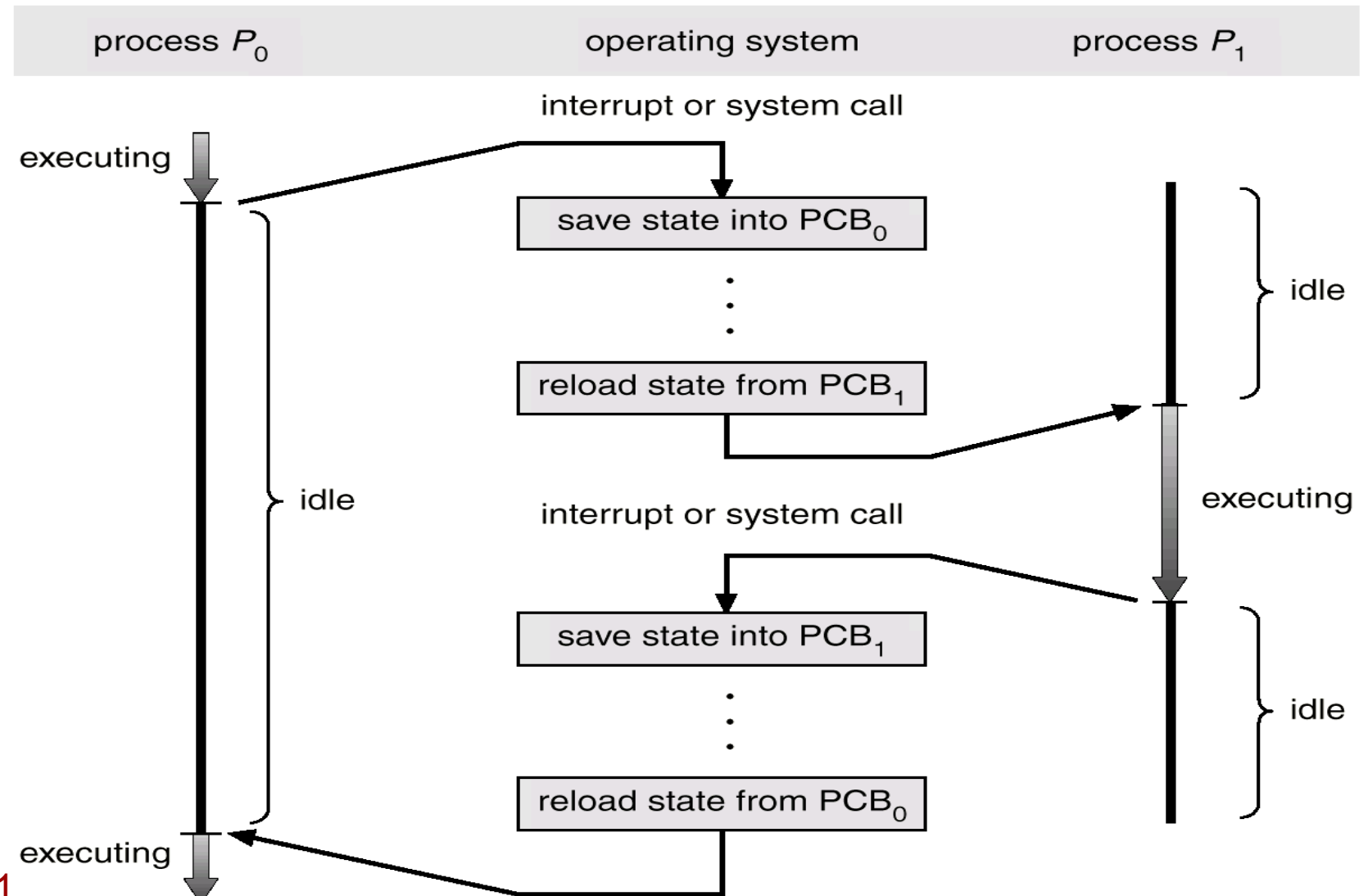
Le informazioni relative ad un processo (il suo stato di esecuzione) sono contenute in una struttura dati apposita.

- Stato
- Program counter
- Registri della CPU
- Informazioni sullo scheduling CPU
- Informazioni sulla gestione della memoria
- Informazioni sullo stato dell' I/O
- Informazioni contabili

pointer	process state
process number	
program counter	
registers	
memory limits	
list of open files	
⋮	

Context Switch

- Context-switch produce overhead.
- Il tempo di switch dipende dal supporto hardware a disposizione.



Operazioni su Processi: Creazione

Il processo è un tipo di dato astratto sul quale sono state definite alcune operazioni fondamentali, la prima è quella di **creazione**.

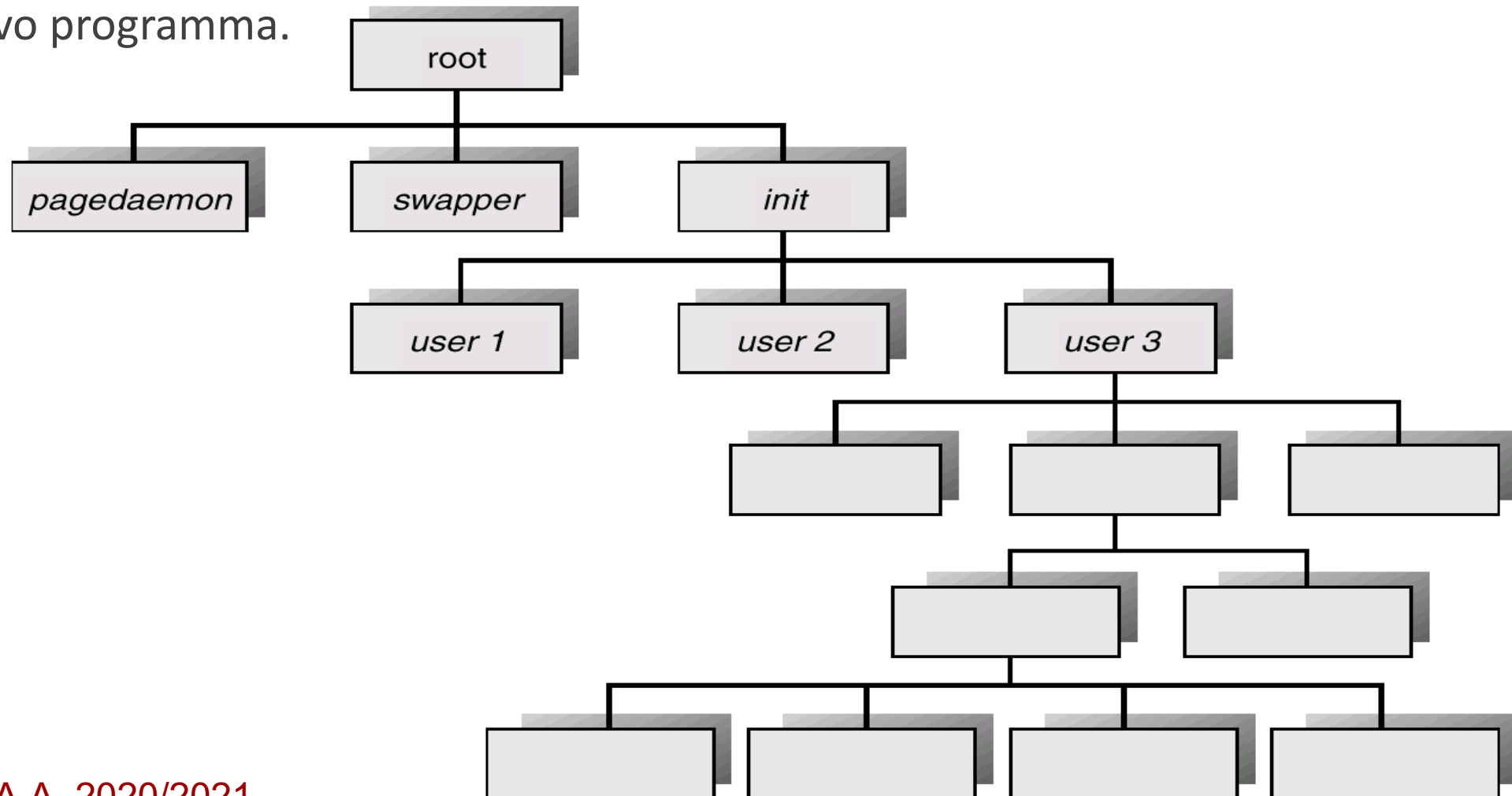
- Un processo padre può creare dei nuovi processi che vengono detti processi figli che a loro volta possono creare altri processi, formando un **albero dei processi**.
- **Condivisione risorse**
 - Genitore e figli condividono tutte le risorse.
 - I figli condividono un sottoinsieme delle risorse del genitore.
 - Genitore e figli NON condividono risorse.
- **Esecuzione**
 - Genitore e figli vengono eseguiti concorrentemente.
 - Il Genitore attende che tutti o alcuni figli terminino.
- **Spazio degli indirizzi**
 - Il figlio è un duplicato del genitore.
 - Nel figlio viene caricato un programma (eventualmente diverso da quello del genitore).



Albero dei Processi in Unix

- In UNIX:

- La system call **fork** crea un nuovo processo
- La system call **exec** sostituisce lo spazio di memoria corrente del processo con un nuovo programma.



Operazioni su Processi: Terminazione

- Un processo esegue l'ultima istruzione e chiede al SO di essere cancellato (**exit**).
 - Può riportare alcuni dati al processo genitore che li riceve con una istruzione (**wait**).
 - Tutte le risorse del processo sono deallocate dal SO.
- Un genitore può terminare i processi figli (**abort**) per diverse ragioni:
 - Il figlio ha ecceduto le risorse allocate
 - Il compito assegnato al figlio non è più necessario
 - Il genitore stesso deve terminare la sua esecuzione
 - e il SO non permette ai figli di continuare dopo la terminazione dei padri.
 - Terminazione a cascata.



Processi e Threads

I PROCESSI E JAVA



- Nel linguaggio di programmazione Java, la programmazione concorrente riguarda principalmente i thread. Tuttavia, anche i processi possono essere gestiti in Java.

Classe **ProcessBuilder**

- Questa classe viene utilizzata per creare processi del sistema operativo.
- Ogni istanza di ProcessBuilder gestisce una raccolta di attributi di processo.
- Il metodo start () crea una nuova istanza di processo con tali attributi. Il metodo start () può essere richiamato ripetutamente dalla stessa istanza per creare nuovi sottoprocessi con attributi identici o correlati.
- <https://docs.oracle.com/javase/8/docs/api/java/lang/ProcessBuilder.html>

Classe **Process**

- Il metodo ProcessBuilder.start () crea un processo nativo e restituisce un'istanza di una sottoclasse di Process che può essere utilizzata per controllare il processo e ottenere informazioni su di esso.
- La classe Process fornisce metodi per eseguire input dal processo, eseguire l'output nel processo, attendere il completamento del processo, controllare lo stato di uscita del processo e distruggere (uccidere) il processo.
- <https://docs.oracle.com/javase/8/docs/api/java/lang/Process.html>



```
public class SimpleProcess {  
    public static void main(String[] args) throws  
        InterruptedException,  
            IOException {  
        ProcessBuilder pb = new ProcessBuilder("ls", "-la");  
        System.out.println("Run command");  
        Process process = pb.start();  
  
        while (process.isAlive()) {  
            System.out.println("The process is still running");  
        }  
        System.out.println("The process stopped. Any errors? " +  
            (process.exitValue() == 0 ? "No" : "Yes"));  
  
        System.out.println("Echo Output:\n" +  
            output(process.getInputStream()));  
    }  
}
```

busy waiting


```
public class SimpleProcess {  
    public static void main(String[] args) throws InterruptedException,  
        IOException {  
        ProcessBuilder pb = new ProcessBuilder("ls", "-la");  
        System.out.println("Run command");  
        Process process = pb.start();  
  
        int errCode = process.waitFor();  
  
        System.out.println("Echo command executed, any errors? " +  
            (errCode == 0 ? "No" : "Yes"));  
  
        System.out.println("Echo Output:\n" +  
            output(process.getInputStream()));  
    }  
}
```

blocking

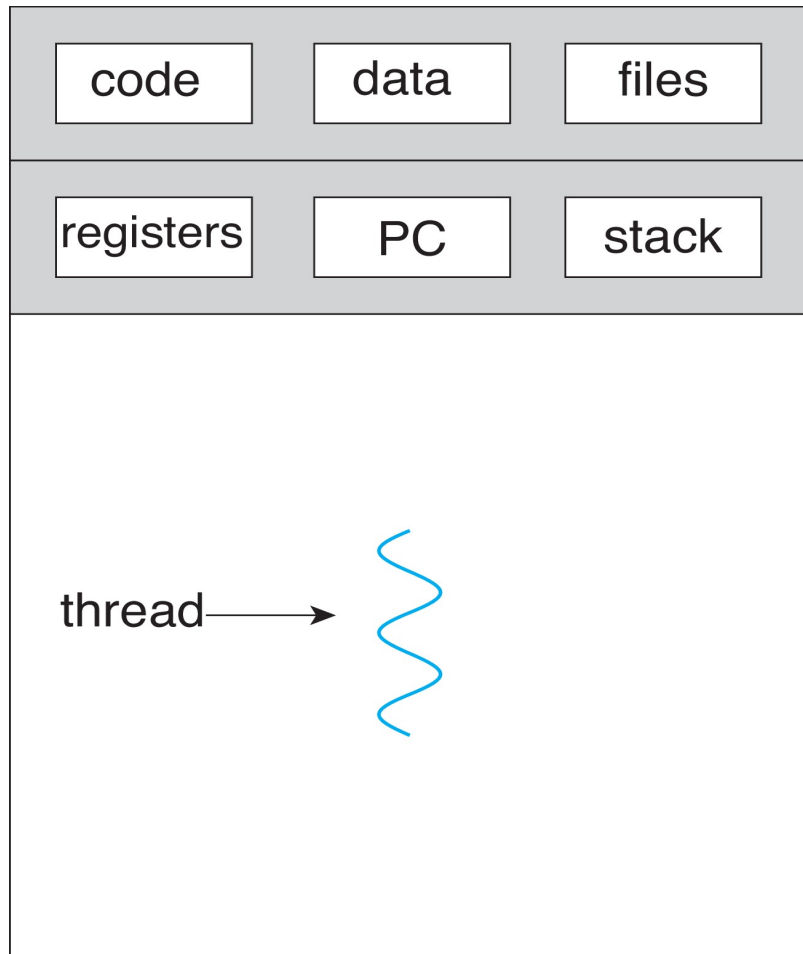
```
public class SimpleProcess {  
    public static void main(String[] args) throws InterruptedException,  
        IOException {  
        ProcessBuilder pb = new ProcessBuilder("ls", "-la");  
        System.out.println("Run command");  
        Process process = pb.start();  
  
        Thread.sleep(1); // Sleep for 0.001 seconds blocking  
        if (process.isAlive()) {  
            process.destroy();  
            System.out.println("The process has been killed. Any errors?" +  
                (process.exitValue() == 0 ? "No" : "Yes"));  
        }  
        else  
        {  
            System.out.println("The process stopped. Any errors? " +  
                (process.exitValue() == 0 ? "No" : "Yes"));  
        }  
  
        System.out.println("Echo Output:\n" +  
            output(process.getInputStream()));  
    }  
}
```



Processi e Threads

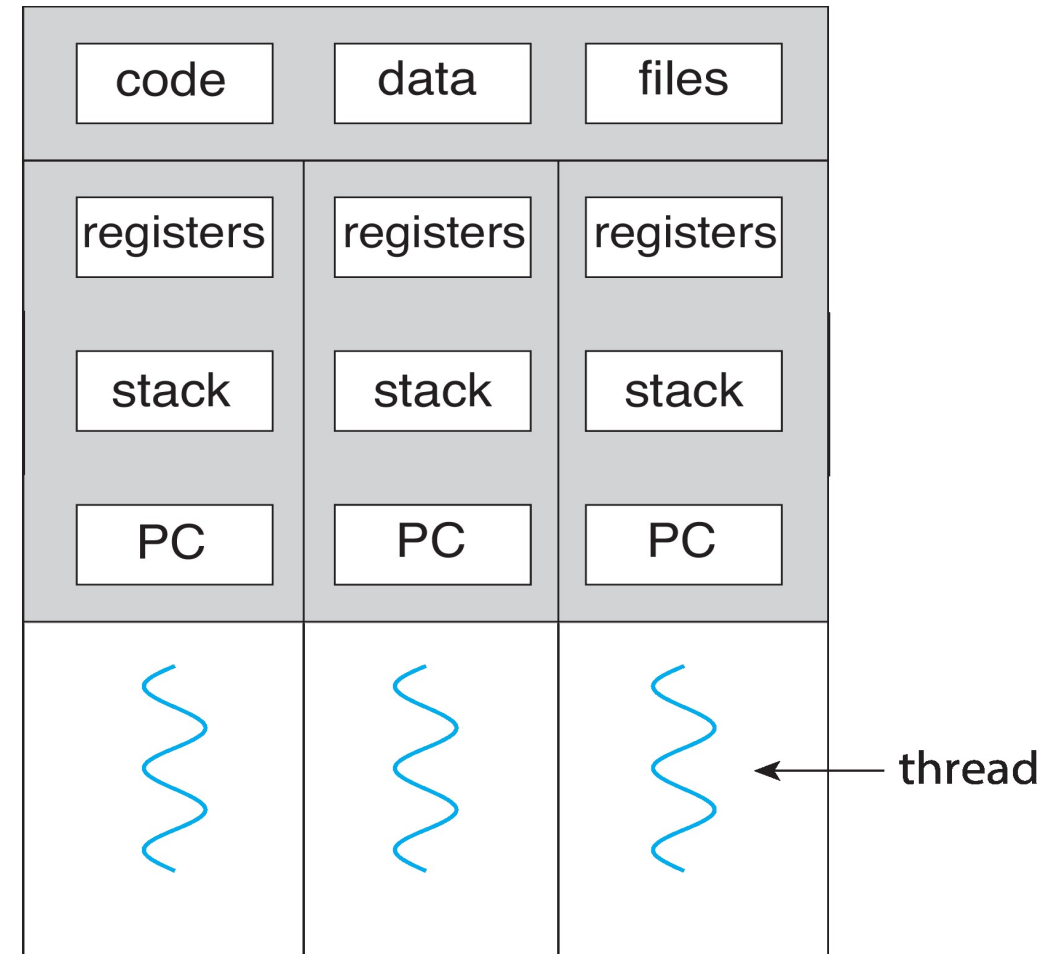
I THREAD E LE OPERAZIONI SUI THREAD

Thread



single-threaded process

Processo a thread singolo



multithreaded process

Processo multithread

- Lo spazio di indirizzamento è condiviso (viene usata la stessa coppia di registri “base e limite” per tutti i thread di un processo).
- Ogni thread tuttavia ha un suo stato (ready, running, waiting), i suoi registri di CPU (stato della computazione), incluso il program counter, stack pointer, ecc., il suo stack.

NON VI È PROTEZIONE FRA THREAD:

- ogni thread può “pasticciare” i dati degli altri (per esempio quelli sullo stack)

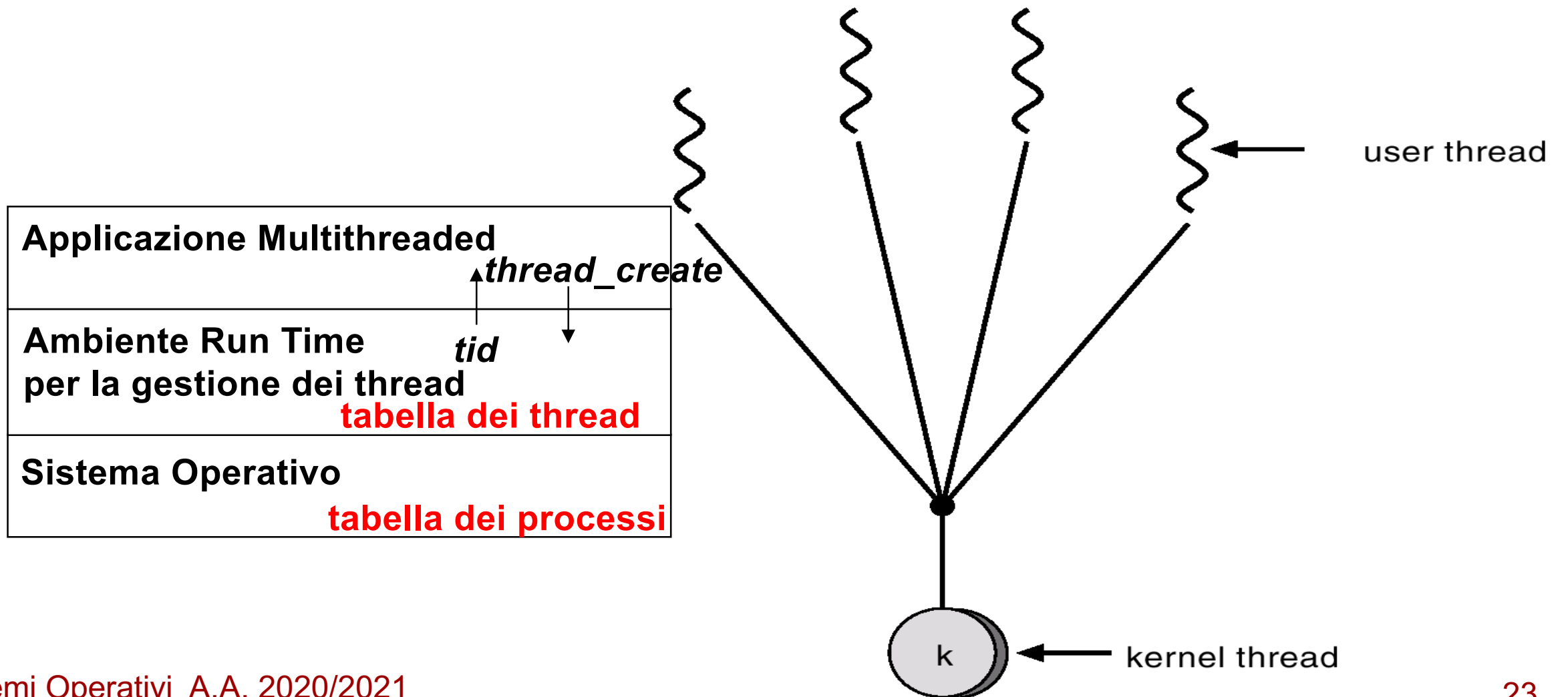
- **Thread a livello utente**
 - i thread sono gestiti da una libreria a livello utente.
 - Esempio: Java Virtual Machine
- **Thread a livello kernel**
 - le funzionalità dei thread sono messe a disposizione e gestite direttamente dal kernel.
 - Esempio: Windows, Linux, Mac OS X, Android

Modelli di Multithreading:

- Molti-a-Uno
- Uno-a-Uno
- Molti-a-Molti

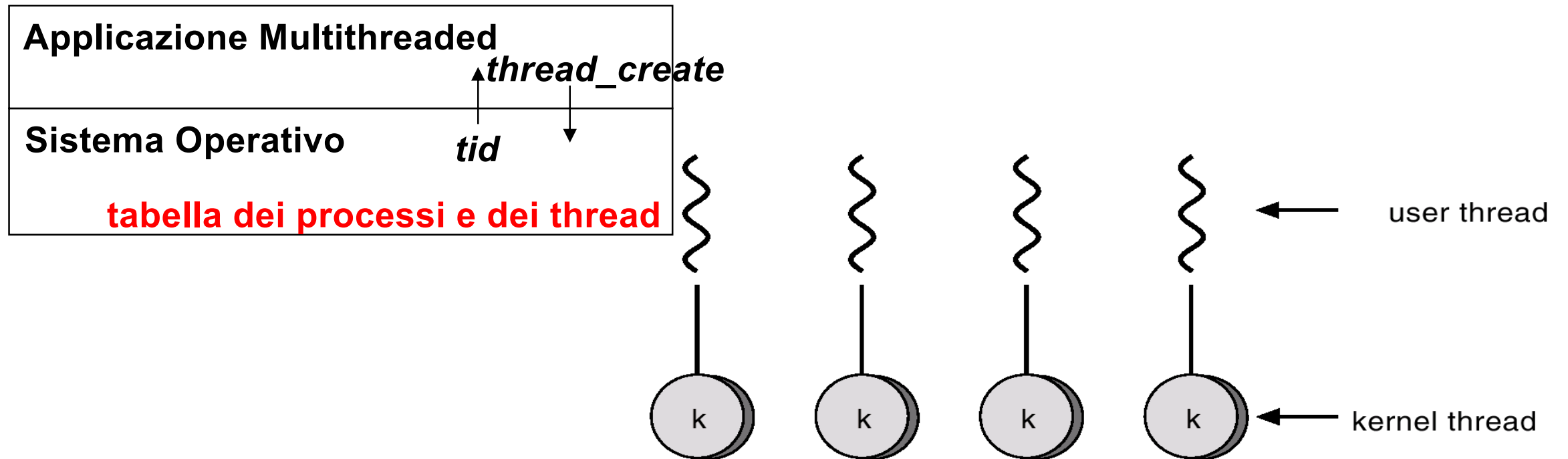
Modello Multi-a-Uno

- Molti thread a livello utente sono mappati ad un singolo thread a livello kernel.
- Viene usato nei sistemi che non forniscono thread a livello utente.



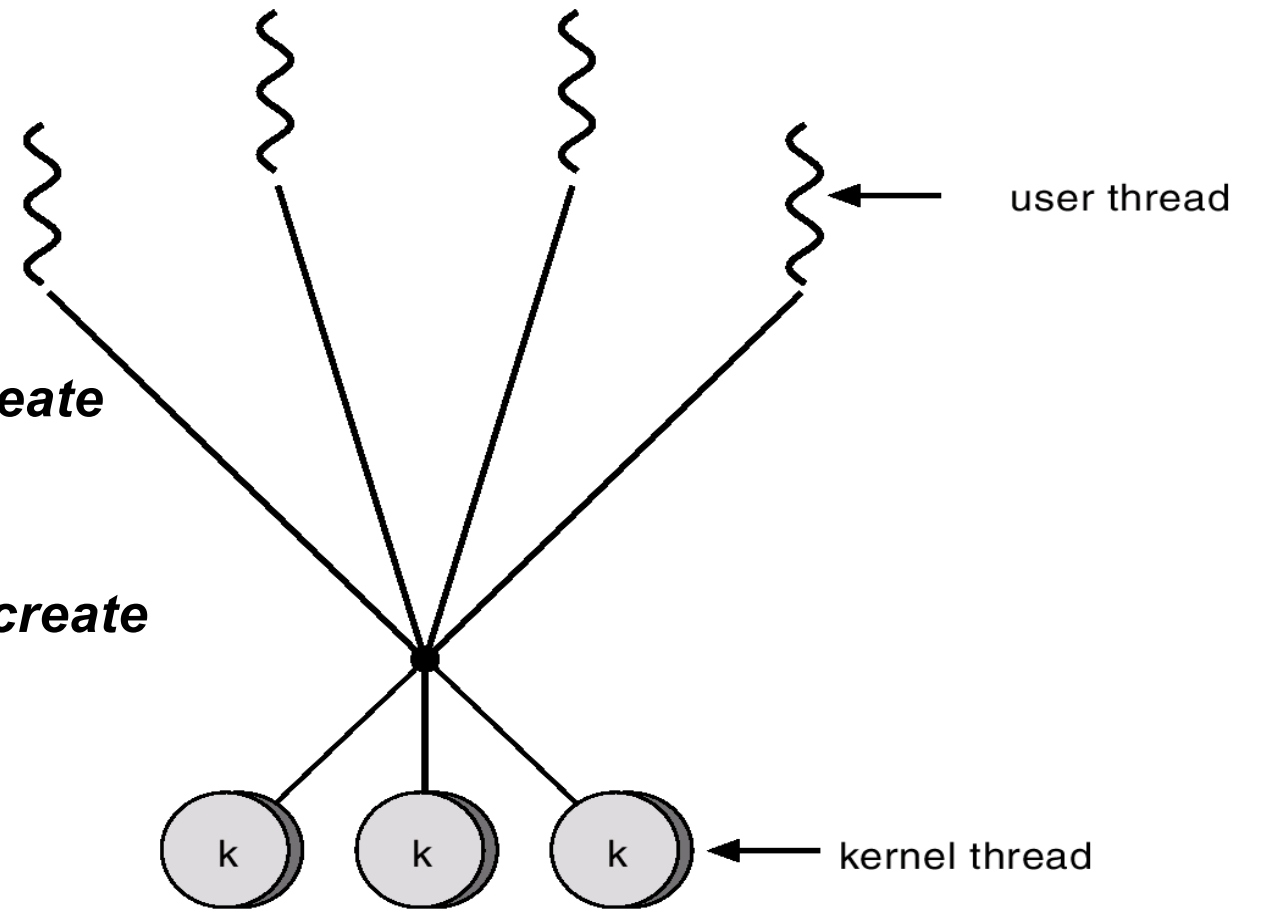
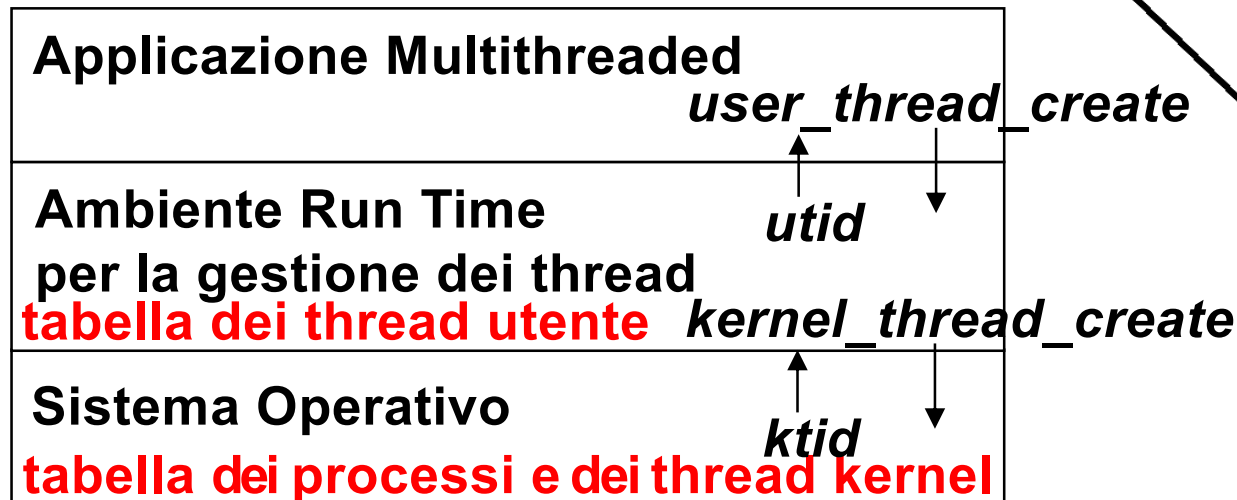
Modello Uno-a-Uno

- Ogni thread a livello utente si mappa ad un thread a livello kernel.
 - Esempio: linux, windows



Modello Multi-a-Molti

- Diversi thread a livello utente sono mappati a diversi thread a livello kernel.
- Questo permette al SO di creare un numero sufficiente di thread a livello kernel.
 - Non molto comune





Processi e Threads

I THREAD E JAVA



```
public class ConcurrentCount {
    public static void main(String[] args) {
        CCounter c = new CCounter();

        // threads t and u, sharing counter
        Thread t = new Thread(c);
        Thread u = new Thread(c);

        t.start(); // increment once
        u.start(); // increment twice

        try { // wait for t and u to terminate
            t.join(); u.join(); } blocking
        catch (InterruptedException e)
        { System.out.println("Interrupted!"); }

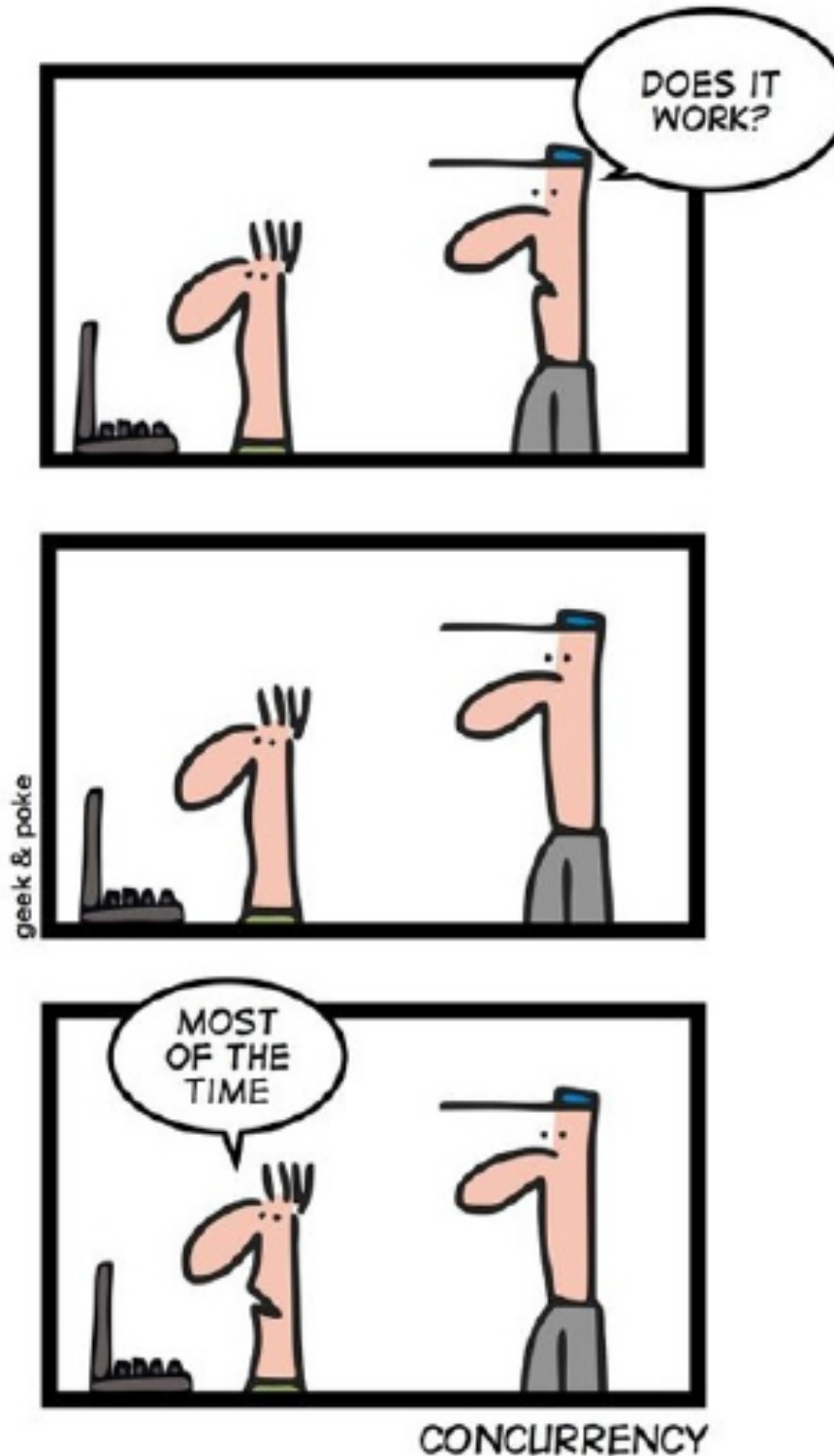
        // print final value of counter
        System.out.println(c.getCounter());
    } }
```

```
public class CCounter
    implements Runnable
{
    private int counter = 0;

    // increment counter by one
    public void run() {
        setCounter(getCounter() + 1);
    }

    // getter
    public int getCounter() {
        return counter;
    }

    //setter
    public void setCounter(int v) {
        counter = v;
    }
}
```



Funziona?

Esempio in azione – una esecuzione

```
1. public class CCounter implements Runnable {  
2.     private int counter = 0;
```

```
    // increment counter by one
```

```
3.     public void run() {  
4.         int tmp = getCounter();  
5.         setCounter(tmp + 1);  
6.     }
```

```
...  
}
```

t		u		shared
pc _t =4;	tmp _t =⊥	pc _u =4;	tmp _u =⊥	counter=0
pc _t =5;	tmp _t =0	pc _u =4;	tmp _u =⊥	counter=0
pc _t =6;	tmp _t =0	pc _u =4;	tmp _u =⊥	counter=1
done		pc _u =4;	tmp _u =⊥	counter=1
done		pc _u =5;	tmp _u =1	counter=1
done		pc _u =6;	tmp _u =1	counter=2
done		done		counter=2

Esempio in azione – un'altra esecuzione

```
1. public class CCounter implements Runnable {  
2.     private int counter = 0;
```

```
    // increment counter by one
```

```
3.     public void run() {  
4.         int tmp = getCounter();  
5.         setCounter(tmp + 1);  
6.     }
```

```
...  
}
```

t		u		shared
pc _t =4;	tmp _t =⊥	pc _u =4;	tmp _u =⊥	counter=0
pc _t =5;	tmp _t =0	pc _u =4;	tmp _u =⊥	counter=0
pc _t =5;	tmp _t =0	pc _u =5;	tmp _u =0	counter=0
pc _t =5;	tmp _t =0	pc _u =6;	tmp _u =0	counter=1
pc _t =6;	tmp _t =0	done		counter=1
done		done		counter=1

Benvenuti nel mondo della concorrenza!!!





Processi e Threads

I TASK DI LINUX

I Task di Linux

- Linux utilizza la stessa rappresentazione interna sia per i processi sia per i thread che in entrambi i casi vengono chiamati **task**.
- L'unica differenza è che un thread è un task che condivide lo stesso spazio degli indirizzi con il genitore mentre il processo ha uno spazio degli indirizzi diverso.
- Questa differenza viene indicata in fase di creazione del task effettuando una system call opportuna
 - **fork** crea un nuovo task con un suo nuovo contesto
 - **clone** crea un nuovo task con una sua identità propria, ma con le strutture dati condivise con il suo genitore
 - L'uso di clone () fornisce a un'applicazione un controllo dettagliato su ciò che è condiviso esattamente tra due thread

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

I Task di Linux

- Linux utilizza la stessa rappresentazione interna sia per i processi sia per i thread che in entrambi i casi vengono chiamati **task**.
- L'unica differenza è che un thread è un task che condivide lo stesso spazio degli indirizzi con il genitore mentre il processo ha uno spazio degli indirizzi diverso.
- Questa differenza viene indicata in fase di creazione del task effettuando una system call opportuna
 - **fork** crea un nuovo task con un suo nuovo contesto
 - **clone** crea un nuovo task con una sua identità propria, ma con le strutture dati condivise con il suo genitore
- Le proprietà di un task linux possono essere classificate in tre categorie:
 - Identità del task
 - Ambiente del task
 - Contesto del task

I Task di Linux

Identità di un Task

- **Process ID (PID)**. Identifica in modo univoco un task. Per esempio, se una applicazione deve effettuare una system call per inviare un messaggio, modificare o attendere un task, il parametro da passare al SO è il PID del task.
- **Credential**. Ad ogni task viene associato uno **user ID** ed uno o più **group ID**. Serve per determinare i diritti di accesso del task a risorse e a file.
- **Personality**. Questa caratteristica non si trova nei tradizionali sistemi Unix. Sotto Linux, ad ogni task viene associato un **personality ID**. Modifica la semantica di certe system call.
Principalmente viene utilizzato da librerie di emulazione per rendere le system call compatibili con determinate caratteristiche di Unix.
- **Namespace**: vista specifica sulla gerarchia del file system
 - La maggior parte dei processi condivide lo spazio dei nomi comune e opera su una gerarchia di file system condivisa
 - Ma ognuno può avere una gerarchia di file system univoca con la propria directory principale e un insieme di file system montati

- Ogni task eredita dal genitore due vettori:
 - Il **vettore degli argomenti** contiene tutti gli argomenti presenti nella linea di comando per avviare il programma. Convenzionalmente il primo argomento è il nome del programma stesso.
Questo è uno strumento per parametrizzare l'esecuzione dei programmi.
 - Il **vettore dell'ambiente** è una lista di coppie del tipo
"NAME=VALUE"
che associa ad ogni variabile di ambiente un valore.
In questo modo ogni task può avere il SO configurato ad hoc, invece di avere una configurazione unica per tutti i task.
 - Inoltre le variabili possono essere utilizzate per passare informazioni da un task ad un altro.
- Far ereditare ad un task l'ambiente del task genitore è un modo per far passare informazioni dal genitore al figlio.

I Processi Linux

Contesto di un task

- Lo stato dell'esecuzione di un task (le informazioni contenute in un descrittore di task).
 - Lo **scheduling context** è la parte più importante. Sono le informazioni necessarie allo scheduler per sospendere e riattivare un task.
 - **Informazioni contabili** su tutte le risorse usate attualmente ed utilizzate fino a quel momento dal task.
 - La **tabella dei file aperti** è un array di puntatori ai **descrittori di file (fd)**
 - Quando si effettuano chiamate di sistema I/O di file, i processi fanno riferimento ai file in base al loro indice in questa tabella
 - Mentre la tabella dei file elenca i file aperti esistenti, il **file system context** si applica alle richieste di apertura di nuovi file
 - Le attuali directory di root e di default da utilizzare per le nuove ricerche di file vengono memorizzate qui
 - La **signal-handler table** indica quali routine (appartenenti allo spazio degli indirizzi del task) devono essere chiamate quando al task arriva uno specifico segnale (signal).
 - La **virtual-memory context** indica l'intero contenuto dello spazio degli indirizzi del task.